

How to use this guide: Every section maps to a layer you add to your agent. Start with Section 1 (The Core Loop) for any new agent. Add sections progressively as your system grows in complexity. Section 8 is the checklist — use it before shipping.

1. The Core Loop

Always start here — every agent runs this pattern

The ReAct Pattern

Think → Act → Observe → Repeat (until done or max_iterations reached)

```
def run_agent(task, tools, max_iterations=20):
    messages = [{"role": "user", "content": task}]
    for _ in range(max_iterations):
        response = llm.call(tools=tools, messages=messages)
        if response.stop_reason == "end_turn":
            return response # Task complete
        result = execute_tool(response)
        messages.append(result) # MUST feed back or agent forgets
    return {"status": "max_iterations_reached"} # safety exit
```

Non-Negotiable Rules

Rule	What it means
Max iteration cap	Always set — never remove. Without it the agent loops forever and costs grow unbounded.
Append tool results	Every tool result must go back into the messages array. Skip this and the agent loses memory of what just happened.
Observe before acting	The agent must read the tool result before deciding the next step. Never auto-proceed.
Single exit condition	stop_reason == "end_turn" means the agent decided it was done. Hitting max_iterations means something went wrong.

2. Tool Design Rules

Tools are the most important part — bad tools break good agents

Rule	What it means
One tool, one job	search_products() not process_everything(). Ambiguous tools cause agents to call the wrong one with high confidence.
Always typed output	Use Pydantic — never return raw dicts or strings. The agent cannot reliably access fields it cannot type-check.
Never raise exceptions	Return ToolResult(success=False, error="..."). Exceptions crash the agent loop.
Idempotent reads	Calling a read tool twice gives the same result. Never modify state in a read tool.
Register centrally	TOOL_REGISTRY = {"name": function}. Agent imports only the registry — never individual tools.

Base Tool Contract — use this for every tool you write

```

from pydantic import BaseModel
from typing import Optional, Any

class ToolResult(BaseModel):
    success: bool
    error: Optional[str] = None

# Every tool follows this exact contract
def my_tool(param: str) -> MyResult:
    try:
        # do work
        return MyResult(success=True, data=result)
    except Exception as e:
        return ToolResult(success=False, error=str(e))

# Central registry – agent imports only this
TOOL_REGISTRY = {
    "search_products": search_products,
    "get_order_details": get_order_details,
    "check_inventory": check_inventory,
    "create_ticket": create_support_ticket,
}

```

3. Reactive vs Planning Agent

Choose the right pattern before writing any code

Situation	Pattern to use
1–2 steps, no dependencies	Reactive agent — simple loop
3+ steps that depend on each other	Planning agent — Planner + Executor
Human needs to see plan first	Planning agent — always
Multi-domain, multi-tool	Planning agent
Simple lookup or single action	Reactive agent

The Task Model — contract between Planner and Executor

```

from pydantic import BaseModel
from typing import Optional, Any

class Task(BaseModel):
    step: int # execution order: 1, 2, 3...
    action: str # human-readable description
    tool: str # must exist in TOOL_REGISTRY
    params: dict # parameters passed to the tool
    depends_on: Optional[int] = None # step number this needs output from
    status: str = "pending" # pending | running | done | failed
    result: Optional[Any] = None # populated by Executor after call

class TaskPlan(BaseModel):
    request: str
    tasks: list[Task]
    approved: bool = False # human gate — must be True to run

```

Planner + Executor Rules

Rule	What it means
Planner never calls tools	It only generates the task list. If the Planner calls tools it becomes a reactive agent.
Executor never decides	It only runs the plan. If Executor is deciding what to do next, the Planner failed.
Validate before handing off	Every Task.tool must exist in TOOL_REGISTRY. Every Task.depends_on must point to a real step.
Retry once on failure	If a tool returns success=False — retry once. If retry fails — set status="failed" and stop that branch.
Pass results between steps	Step 2 that depends_on step 1 gets step 1's result injected into its params at runtime.

4. Memory — Three Layers

Add only the layers your agent actually needs

Layer	What it is	When to add it
In-Context	The messages array. Fast, immediately available. Gone when session ends.	Always — this is your base loop.
Vector DB	ChromaDB / Pinecone. Persists between sessions. Semantic search.	When the agent needs to remember across sessions.
RAG Knowledge	Your domain docs embedded and searchable. Agent retrieves before generating.	When the agent needs domain knowledge the LLM was not trained on.

RAG Pipeline — 4 steps

```
# Step 1: Chunk documents (200-400 tokens, 15-20% overlap)
def chunk_text(text, chunk_size=300, overlap=60):
    words = text.split()
    chunks = []
    for i in range(0, len(words), chunk_size - overlap):
        chunks.append(" ".join(words[i : i + chunk_size]))
    return chunks

# Step 2: Embed and store in ChromaDB
collection.add(documents=chunks, ids=[f"doc_{i}" for i in range(len(chunks))])

# Step 3: Retrieve before every LLM call
def retrieve_context(query, n_results=3):
    results = collection.query(query_texts=[query], n_results=n_results)
    return "\n\n".join(results["documents"][0])

# Step 4: Inject into prompt
context = retrieve_context(user_question)
prompt = f"Context:\n{context}\n\nQuestion: {user_question}"
```

Key rule: Always retrieve before generating. Wrong chunks produce confident wrong answers — which is worse than no answer. Test retrieval quality with every expected question before shipping.

5. Multi-Agent Architecture

When one agent gets too complex, split by responsibility

Worker + Coordinator Pattern

```

# Each worker has one responsibility
class AgentMessage(BaseModel):
    task_id: str
    worker: str # "researcher" | "writer" | "qa" | "deploy"
    instruction: str
    context: dict = {}

class WorkResult(BaseModel):
    task_id: str
    worker: str
    success: bool
    output: Any = None
    error: Optional[str] = None

# Coordinator dispatches and collects
async def run_coordinator(topic: str) -> dict:
    memory = SharedMemory()

    research = await run_worker(AgentMessage(worker="researcher", ...))
    memory.store("research", research.output)

    draft = await run_worker(AgentMessage(worker="writer", ...))
    memory.store("draft", draft.output)

    qa = await run_worker(AgentMessage(worker="qa", ...))
    if qa.output["passed"]:
        deploy = await run_worker(AgentMessage(worker="deploy", ...))

```

Rule	What it means
SharedMemory connects workers	Workers read and write to a shared context store — never call each other directly.
Coordinator never does the work	It only dispatches, collects results, and makes routing decisions.
Each worker is independently testable	You can run <code>researcher_worker("topic")</code> alone. If it needs the coordinator to work, it is too coupled.
QA gates progression	If QA score < threshold, loop back to Writer. Only proceed to Deploy on explicit pass.

6. External Integrations — MCP Pattern

Put external services behind a standard server

```
# MCP Server (FastAPI) — two endpoints, that is all
@app.post("/tools/list")
async def list_tools():
    return {"tools": TOOLS} # agent discovers capabilities

@app.post("/tools/call")
async def call_tool(req: ToolCallRequest):
    if req.tool_name not in TOOL_REGISTRY:
        raise HTTPException(404, f"Tool {req.tool_name} not found")
    try:
        result = TOOL_REGISTRY[req.tool_name](**req.params)
        return result.model_dump()
    except TypeError as e:
        return {"success": False, "error": f"Invalid params: {str(e)}"}

# Agent workflow — always in this order
async def run_deploy_agent():
    client = MCPClient("http://localhost:8000")

    # 1. Connect
    if not await client.connect():
        return DeployResult(success=False, error="MCP server unreachable")

    # 2. Discover — never hardcode tool names
    tools = await client.list_tools()

    # 3. Validate required tools exist
    required = {"publish_product", "update_seo", "notify_team", "create_pr"}
    found = {t.name for t in tools}
    if not required.issubset(found):
        return DeployResult(success=False, error=f"Missing tools: {required - found}")

    # 4. Call sequentially
    result = await client.call_tool("publish_product", sku=sku, price=price)
```

Why MCP matters: Agent discovers capabilities at runtime — adding a new integration means adding it to the server, not changing agent code. The agent always calls `list_tools()` first; tool names are never hardcoded in agent logic.

7. Production Layer

Four features that turn a prototype into a production system

7a — Tracing (Langfuse or equivalent)

```

from langfuse import Langfuse
langfuse = Langfuse(public_key=..., secret_key=...)

async def run_agent(topic):
    trace = langfuse.trace(name="agent_run", metadata={"topic": topic})

    span = trace.span(name="worker_dispatch", input={"topic": topic})
    result = await run_worker(topic)
    span.end(output=result, level="DEFAULT") # level="ERROR" on failure

    langfuse.flush() # always flush before returning
    return result

```

7b — Output Guards (Pydantic validation)

```

from pydantic import BaseModel, Field, validator
from typing import List, Optional

class QAOutput(BaseModel):
    qa_score: float = Field(ge=0.0, le=1.0) # rejects 1.7 automatically
    notes: List[str]
    passed: bool
    rewrite_hint: Optional[str] = None

# Apply after every LLM call
try:
    validated = QAOutput(qa_score=score, notes=notes, passed=score >= 0.7)
    return WorkResult(success=True, output=validated.model_dump())
except ValidationError as e:
    end_span(span, output=str(e), level="ERROR") # log to trace
    return WorkResult(success=False, error=f"Guard failed: {e}")

```

7c — Human Approval Gate

```

async def request_approval(prompt: str, preview: str = None) -> bool:
    print("=" * 60)
    print("HUMAN APPROVAL REQUIRED")
    print("=" * 60)
    print(prompt)
    if preview:
        print(preview[:400])
    response = input("Type yes to proceed, anything else to abort: ")
    return response.strip().lower() == "yes"

# Use before any irreversible action
if not await request_approval(f"Deploy? QA score: {qa_score:.2f}", draft):
    return {"status": "aborted", "reason": "Human rejected"}
# Deploy only runs here – after explicit yes

```

7d — Model Routing


```
# Route each worker to the right model
WORKER_MODELS = {
    "researcher": "claude-haiku-4-5", # retrieval – fast, cheap
    "writer": "claude-sonnet-4-5", # generation – quality matters
    "qa": "claude-sonnet-4-5", # scoring – nuance matters
    "coordinator": "claude-haiku-4-5", # routing decisions – simple logic
    "deploy": "claude-haiku-4-5", # tool selection – deterministic
}

def get_model(worker: str) -> str:
    return WORKER_MODELS.get(worker, "claude-sonnet-4-5")

# Replace every hardcoded model string
response = client.messages.create(model=get_model("researcher"), ...)
# Rule: use the smallest model that meets the quality bar for that step
```

8. Implementation Checklist

Run through this before shipping any agent

Design Phase

- What is the single responsibility of this agent?
- What tools does it need? Is each tool doing exactly one thing?
- Does any step depend on a previous step's output? → Use planning agent
- Does a human need to approve before any irreversible action?
- What is the quality bar that determines success vs failure?

Build Phase

- Every tool returns a typed Pydantic model — not a raw dict or string
- Every tool catches exceptions and returns `ToolResult(success=False, error="...")`
- Tool results are appended to the messages array after every call
- Max iteration cap is set and cannot be removed
- Each tool is tested in isolation before adding to `TOOL_REGISTRY`
- Planner validates all tool names against `TOOL_REGISTRY` before handing off
- Executor checks `depends_on` before each step and injects prior results

Production Phase

- Tracing: every agent run produces a named trace visible in the dashboard
- Each worker dispatch is a separate span with input and output captured
- Guards: LLM outputs are validated through Pydantic models before propagating
- Guard failures log `level="ERROR"` to the trace — never silently swallowed
- Gate: human approval before any irreversible action (deploy, publish, send)
- Routing: cheapest model that meets quality bar is assigned to each worker
- `langfuse.flush()` (or equivalent) is called before every return
- Gate abort test: typing "no" returns `status="aborted"` and deploy step does not run

9. The Decision Framework

Pick the right pattern before writing a single line

Situation	Pattern to use
Simple task, one step	→ Single tool call — no loop needed
Multi-step, known sequence	→ Planner + Executor
Needs domain knowledge	→ Add RAG layer on top of any pattern
Multiple specialisations	→ Multi-agent with coordinator
External services / APIs	→ MCP server pattern

Going to production	→ Tracing + Guards + Gate + Routing
Everything above	→ Full 5-agent production system

The core principle: Start with the simplest thing that works. A reactive agent is better than a broken planner. A working prototype with no tracing is better than no prototype. Add layers only when the simpler version breaks.

"A reactive agent tries. A planning agent succeeds. A production agent is observed, guarded, and approved."